

Hands-On GNN Chapter 3

A7610 그래프신경망특론

March 18, 2026

Contents

1 문제의식	1
2 Word2Vec 직관	2
3 DeepWalk	2
4 Karate Club 실습	3
5 정리	4

목차

Contents

1 문제의식

왜 Node Representation이 필요한가?

- 많은 그래프 ML 문제는 노드를 고정 길이 벡터로 바꾼 뒤 더 쉽게 풀린다.
- 목표: 구조적으로 비슷한 노드는 임베딩 공간에서도 가깝게 놓기
- 이런 벡터는 노드 분류, 링크 예측, 군집화의 공통 입력이 된다.
- Chapter 3는 GNN 이전 단계의 대표 기법인 **DeepWalk**를 다룬다.

$$\text{graph } G = (V, E) \rightarrow \mathbf{z}_v \in \mathbb{R}^d$$

핵심 아이디어: 문장 임베딩을 그래프에 옮기기
NLP 관점

- 단어는 문맥 속에서 의미를 얻는다.
- Skip-gram은 주변 단어를 잘 맞추는 벡터를 학습한다.

그래프 관점

- 노드는 이웃과 경로 속에서 역할을 가진다.
- random walk를 문장처럼 보고 같은 학습기를 적용한다.

$$\text{word sequence} \Rightarrow \text{random-walk sequence}$$

2 Word2Vec 직관

노트북 1부: Word2Vec로 임베딩 감각 익히기

- 먼저 텍스트를 작은 단어 시퀀스로 만들고,
- `Word2Vec(..., sg=1)`로 skip-gram 임베딩을 학습한다.
- 주변 단어를 예측하도록 학습된 임베딩은 동시출현 구조를 압축한다.

```
model = Word2Vec([text],
                  sg=1,
                  vector_size=10,
                  min_count=0,
                  window=2,
                  workers=1,
                  seed=0)
```

Skip-gram이 학습하는 것

- 중심 단어 w_t 가 주어졌을 때 주변 단어를 맞추는 확률을 최대화
- window가 클수록 더 넓은 문맥을 반영
- 결국 비슷한 문맥에 있는 단어들이 비슷한 벡터를 갖게 된다.

$$\max \sum_{t=1}^T \sum_{-c \leq j \leq c, j \neq 0} \log p(w_{t+j} | w_t)$$

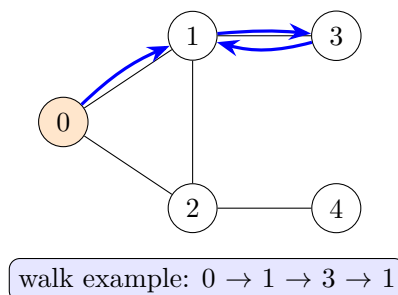
DeepWalk로의 연결

단어 대신 노드, 문장 대신 random walk를 넣으면 같은 목적함수를 그대로 사용할 수 있다.

3 DeepWalk

그래프에서 문장을 만드는 법: Random Walk

- 시작 노드 하나를 정하고
- 현재 노드의 이웃 중 하나를 무작위로 선택하며 이동
- 일정 길이만큼 반복하면 노드 시퀀스 하나가 생긴다.



노트북의 random walk 구현

```
def random_walk(start, length):
    walk = [str(start)]
    for i in range(length):
        neighbors = [node for node in G.neighbors(start)]
        next_node = np.random.choice(neighbors, 1)[0]
        walk.append(str(next_node))
        start = next_node
    return walk
```

- 반환값을 문자열로 두는 이유는 Word2Vec 입력 형식과 맞추기 위해서다.
- 한 번의 walk는 noisy하지만, 여러 번 반복하면 지역 구조가 안정적으로 드러난다.

DeepWalk 알고리즘 요약

1. 모든 노드를 시작점 후보로 둔다.
2. 각 노드에서 여러 개의 random walk를 생성한다.
3. 생성된 walk 집합을 말뭉치(corpus)로 본다.
4. Skip-gram / Word2Vec으로 노드 임베딩을 학습한다.

$$\text{walks} = \{(v_1, v_2, \dots, v_L)\} \Rightarrow \text{Skip-gram training}$$

해석

같은 walk 문맥에 자주 등장하는 노드는 구조적으로 관련된 노드로 간주된다.

하이퍼파라미터가 의미하는 것

- **walk length**: 한 시퀀스가 보는 구조 범위
- **walks per node**: 각 노드를 얼마나 자주 샘플링할지
- **window**: 문맥으로 인정할 주변 거리
- **embedding dimension**: 표현 용량

```
model = Word2Vec(walks,
                  hs=1,
                  sg=1,
                  vector_size=100,
                  window=10,
                  workers=1,
                  seed=1)
```

4 Karate Club 실습

노트북 2부: Karate Club 그래프

- `networkx.karate_club_graph()`는 고전적인 커뮤니티 분할 예제다.
- 라벨은 Mr. Hi와 Officer 두 클럽으로 구성된다.
- 구조만으로도 어느 정도 커뮤니티 분리가 가능하다는 점을 보여주기 좋다.

```
G = nx.karate_club_graph()
labels = []
for node in G.nodes:
    label = G.nodes[node]['club']
    labels.append(1 if label == 'Officer' else 0)
```

실험 설정

- 노드마다 80개의 walk 생성
- 각 walk 길이: 10
- 임베딩 차원: 100
- 학습 후 세 가지를 확인
 - 유사 노드 검색
 - t-SNE 2차원 시각화
 - RandomForest 기반 노드 분류

의미

DeepWalk는 비지도 표현학습이고, 분류기는 그 위에 얹는 다운스트림 태스크다.

유사도와 시각화 해석

- `most_similar('0')`은 노드 0과 비슷한 문맥을 가진 노드를 찾는다.
- `wv.similarity('0', '4')`는 두 노드 임베딩의 코사인 유사도를 본다.
- t-SNE는 고차원 임베딩이 라벨별로 어느 정도 분리되는지 직관을 준다.

```
nodes_wv = np.array([model.wv.get_vector(str(i))
                     for i in range(len(model.wv))])
tsne = TSNE(n_components=2,
            learning_rate='auto',
            init='pca',
            random_state=0).fit_transform(nodes_wv)
```

다운스트림 분류 결과의 의미

- 학습 데이터는 12개 노드만 사용
- 테스트 데이터는 나머지 22개 노드
- 임베딩만으로도 커뮤니티 분류가 꽤 잘 되면,
- DeepWalk가 구조 정보를 유용한 feature로 압축했다는 뜻이다.

```
clf = RandomForestClassifier(random_state=0)
clf.fit(nodes_wv[train_mask], labels[train_mask])
y_pred = clf.predict(nodes_wv[test_mask])
acc = accuracy_score(y_pred, labels[test_mask])
```

5 정리

Chapter 3 핵심 정리

- DeepWalk는 NLP의 Word2Vec을 그래프 random walk에 이식한 방법이다.
- 노드 임베딩은 비지도 방식으로 학습되고, 이후 분류/예측 태스크에 재사용된다.
- 좋은 임베딩은 같은 커뮤니티 혹은 비슷한 구조적 역할의 노드를 가깝게 둔다.
- 이는 이후 GCN, GraphSAGE 같은 GNN의 메시지 패싱 학습으로 자연스럽게 이어진다.

다음 단계에서 생각할 질문

- 단순 random walk가 DFS/BFS 성향을 조절하지 못하는 한계는?
- 노드 feature가 있을 때는 DeepWalk만으로 충분한가?
- 비지도 임베딩과 end-to-end GNN의 장단점은 무엇인가?